# SemDisc: An End-to-End Query-by-Example Semantic Join Discovery System

Mir Mahathir Mohammad
University of Utah
mahathir.mohammad@utah.edu

El Kindi Rezig
University of Utah
elkindi.rezig@utah.edu

## Abstract

Discovering joinable tables in data lakes is essential for constructing datasets. However, tables often represent equivalent concepts using different syntactic forms, and meaningful join paths frequently traverse intermediate tables that share no direct overlap with the user's query. We demonstrate SemDisc, an interactive system that enables users to discover join paths in schema-less data lakes through a query-by-example interface. Given a table with example values and optional semantic annotations, SemDisc retrieves top-K join paths that unify both traditional equi-joins and semantic joins. Our system leverages locality-sensitive hashing to efficiently approximate semantic joinability at scale, employs large language models to infer column semantic types, and introduces an index that enables fast retrieval of high-quality join paths. The demonstration will allow SIGMOD attendees to upload and preprocess data lakes, explore the resulting join graph and indexed paths, submit query tables, and interactively examine how retrieved join paths satisfy their specified examples. Attendees will gain hands-on experience with semantic join discovery across multiple benchmark datasets. A companion video is available at [3].

## 1 Introduction

Tabular data lakes [1, 7] pose significant challenges for data discovery due to the absence of global schemas and Primary Key–Foreign Key relationships, making it difficult for users to determine *which* tables are joinable and *how* to join them to build a dataset of interest. While join discovery systems [9, 11] have emerged to detect joinable tables through overlapping column values or semantic similarity [9], they fall short in query-by-example data discovery scenarios, as demonstrated in the following example:

**Example:** Consider a data scientist querying the MIT Data Warehouse [2](Figure 1 ②) with example values and column headers (Figure 1 ①). Constructing the desired dataset requires addressing three key challenges: (1) identifying hybrid join paths that contain both semantic joins (for columns representing the same concept differently, e.g., "MATS SCI & ENG" vs. "Materials Science and Engineering") and equi-joins; (2) discovering join paths with hidden tables (Figure 1 ③)- intermediate tables that don't overlap with the values in the query table but are essential for linking relevant tables; and (3) ensuring semantic tuple matching, where the results include tuples that are semantically similar to the example tuples, in addition to producing other tuples that were not specified in the query table (Figure 1 ④).

We present a demonstration of our system SemDisc [15], a query-by-example join discovery system that addresses the above challenges through: (1) scalable hybrid join path discovery via data sketches to encode semantic and equi-joins in a join graph, (2) an Inverted Join Path Index for efficient retrieval of paths including hidden tables and (3) heuristics to ensure joined results contain tuples semantically matching the user's examples.

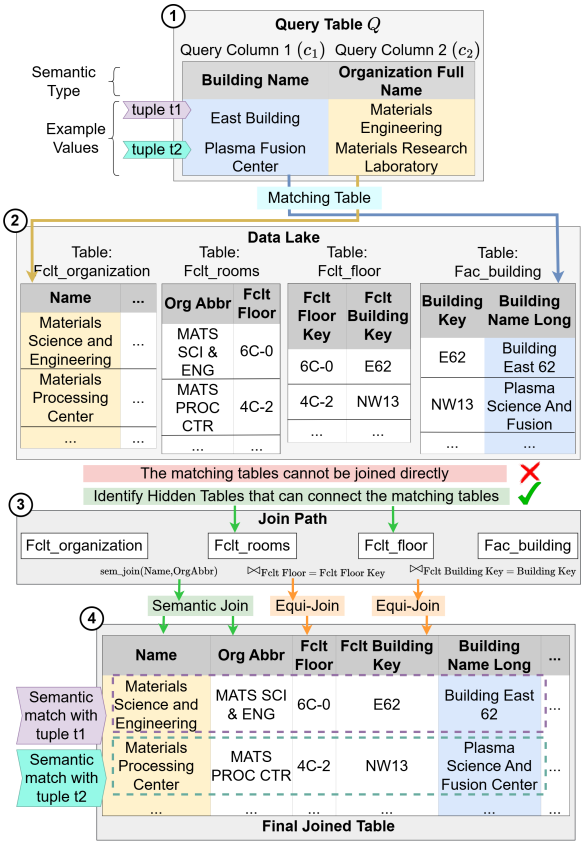


**Figure 1: Example of finding join paths from a query table.**

*Related Work.* There have been several data discovery systems that were proposed in the past few years [7]. We focus the discussion on those that support join discovery. DeepJoin [10] and WarpGate [6] focus on semantic column matching using embeddings but do not construct hybrid join paths with tuple matching for an example query table. Ver [12] and DICE [17] are query-by-example systems that discover join paths using MinHash-based Jaccard similarity; however, they are restricted to equi-joins and do not support semantic joins. Snoopy [13] learns embeddings that capture semantic, rather than syntactic, joinability and retrieves the top-k semantically joinable columns. SemDisc extends this idea by enabling tuple-level semantic matching and hidden-table discovery.

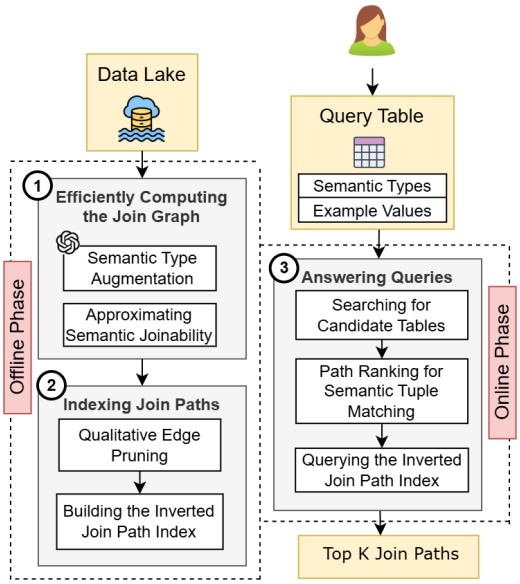


**Figure 2: Overview of SemDisc 's architecture. The offline phase constructs and indexes a join graph over the data lake, while an online phase efficiently retrieves and ranks the top-K join paths for a given query table.**

## 2 System Overview

Figure 2 illustrates the overall architecture of SemDisc. SemDisc operates in two phases: **Offline phase:** In this phase, SemDisc (1) computes a join graph (Figure 2 ①) by ingesting a data lake, using an LLM to annotate semantic types of columns, computing semantic joinability between columns and building a join graph with semantic and equi-join edges; (2) indexes join paths (Figure 2 ②) by applying pruning to retain the best join edges, and constructing an Inverted Join Path Index. **Online phase (Figure 2 ③):** In this phase, users submit a query table with example values, SemDisc matches these against data lake tables using semantic types, applies heuristics to match query tuples to join paths, and retrieves the top-k join paths from the index.

### 2.1 Efficiently Computing the Join Graph

The central data structure used by SemDisc to capture semantic and equi-joinability across a data lake is the *join graph*. In this graph, nodes represent tables in the data lake, while edges encode potential join relationships between pairs of columns drawn from two tables. Each edge is assigned a weight that quantifies the *strength* of joinability between the corresponding columns, measured by the extent of value overlap.

*2.1.1 Semantic Type Augmentation.* To address the challenge of columns sharing overlapping values but referring to different real-world entities (e.g., 'student_id' vs. 'part_id')—especially when column headers are not descriptive— SemDisc employs semantic type annotation across all data lake columns using LLMs. The system prompts GPT-4o [4] with sample values from each column along with co-occurring columns from the same table for context, and the LLM returns suggested semantic types in a structured JSON format.

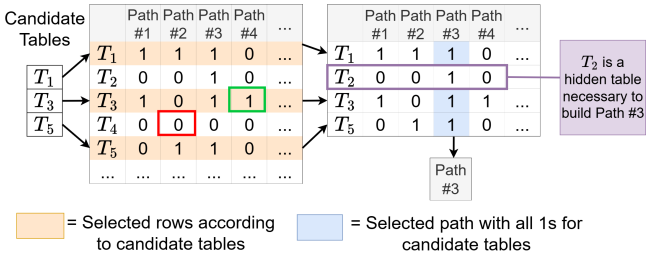


**Figure 3: Retrieving join paths by querying the Join Path Index**

The prompt consists of three parts: an instruction explaining the task, a specified output format listing column IDs paired with their semantic types, and the input columns themselves, each identified by their header and up to 20 of their most frequently occurring values. For details on the prompt template, refer to [15].

*2.1.2 Approximate Semantic Joinability:* Detecting joinability between every pair of columns is expensive. This is why SemDisc employs compact data sketches. Instead of operating on raw column values, each column is summarized using a sketch that captures its value distribution and semantic characteristics. These summaries enable fast and scalable estimation of value overlap between columns, allowing SemDisc to efficiently prune unlikely join candidates. This process is summarized as follows:

**1. SimHash Generation:** This step converts textual column values into vector embeddings using a pre-trained language model (we use SBERT [16]), then hashes them using locality-sensitive hashing (SimHash [15]). Similar values (e.g., "Materials Science and Engineering" and "MATS SCI & ENG") get the same SimHash, reducing semantic matching to simple equality comparison.

**2. MinHash Signatures:** This step creates compact, fixed-size summaries of each column's SimHash values to enable fast set comparison using MinHash [5].

**3. Jaccard Similarity:** Finally, SemDisc compares MinHash signatures between column pairs using Jaccard similarity, which generates the weight of the join edges. Therefore, both semantic and equi-join edges in the join graph share the same weight metric. The resulting join graph connects tables with weighted edges representing joinability between column pairs.

### 2.2 Indexing Join Paths

As we show in [15], join graphs can contain millions of edges, many of which are spurious. SemDisc mitigates this by pruning low-joinability edges and indexing only join paths that consist of high-joinability edges and are estimated—via cardinality estimation—to produce non-empty results.

*2.2.1 Qualitative Edge Pruning.* For each pair of tables, SemDisc filters edges in two steps:

- **Semantic type similarity:** This step prunes edges between columns whose semantic types are not sufficiently similar. This is determined by computing the cosine similarity between their semantic type embedding vectors and discarding edges that fall below a predefined threshold.

- **Value joinability:** In this step, SemDisc only keeps edges meeting a user-defined joinability threshold, then retrains only the single top-ranked edge having the highest weight per table pair.

*2.2.2 Building the Inverted Join Path Index.* Given a set of candidate tables that have value overlap with the query table's columns (the retrieval process is described in Section 2.3), SemDisc needs to find all join paths that contain those tables. Rather than a naive linear search, SemDisc builds an Inverted Join Path Index- a binary matrix where rows represent tables in the data lake, columns represent join paths and a cell value of 1 indicates the table appears in the path; 0 otherwise.

SemDisc constructs the matrix with $|\mathcal{D}| \times |\mathcal{P}|$ zeros where $|\mathcal{P}|$ denotes the count of paths. For each table in each join path, SemDisc sets the corresponding cell to 1. Figure 3 shows an example of the Inverted Join Path index. To find paths containing tables $T_1, T_3, T_5$- SemDisc first selects the rows for those tables, identifies columns where all selected rows contain 1s and returns the corresponding join paths. As shown in the figure, a matching path (e.g. Path 3) may include additional hidden tables (e.g. $T_2$) that aren't candidates but are necessary intermediates to complete the join path.

## 2.3 Answering Queries

This section explains how SemDisc processes user queries in the online phase- specifically, how it efficiently identifies the most relevant join paths and ranks them for a given query table $Q$.

**Searching for Candidate Tables.** For each column in a query table $Q$, the system finds the most relevant data lake columns as follows: (1) SemDisc uses a Hierarchical Navigable Small World (HNSW) [14] index—shown to support highly efficient approximate nearest neighbor (ANN) search—over semantic type embeddings to retrieve the top-matching columns. (2) Then SemDisc filters the results by checking if the query values' SimHashes are contained within each candidate column's SimHashes, ensuring semantic similarity of example values. (3) Finally, SemDisc combines all candidate column lists to produce candidate join paths.

**Path Ranking for Semantic Tuple Matching.** To avoid exhaustively materializing joins, SemDisc ranks candidate join paths based on their potential to produce valid semantic tuple matches. For each candidate set of columns, the system groups columns by their source tables and evaluates whether query tuples can be consistently matched across multiple columns within the same table. Candidate sets that fail to support such partial tuple matches are pruned early, while those that do are assigned higher scores reflecting their matching potential. The resulting ranked list of candidate sets defines a compact and high-quality join path search space, which is then used to query the Inverted Join Path Index.

**Querying the Inverted Join Path Index.** Given the final set of candidate tables, SemDisc queries the Inverted Join Path Index to efficiently retrieve valid join paths. For each candidate set, the system identifies compatible join paths shared across the involved tables. The top-$K$ join paths are then selected and materialized for query execution.

# 3 Demonstration Plan

We demonstrate SemDisc on four data lakes: (1) **U.S. Fish and Wildlife Service (collected from data.gov [1])**, containing 246 tables about ecological records across the U.S.; (2) **Centers for Disease Control and Prevention (collected from data.gov [1])**, containing 467 tables about public health records on disease, mortality, and drug usage; (3) **SchemaPile [8]**, comprising 34.9K tables; and (4) **LakeBench [7]**, a data discovery benchmark containing ground-truth joins between tables.

*Demonstration Outline.* Our demo places participants in the role of a data scientist seeking to construct a dataset by providing a query table. We demonstrate two scenarios:

## 3.1 Scenario 1: Preprocessing the Data Lake

In this scenario, participants dive into the system's internals, examining how the data lake is processed and how join paths are indexed offline.

(1) **Uploading and Preprocessing.** Participants upload a data lake containing CSV files through the SemDisc web interface. System hyperparameters (Figure 4 ❶) such as SimHash size, join threshold, MinHash function count, semantic type similarity, and maximum path length are exposed to users, enabling them to configure the index construction. These hyperparameters control how much pruning is done on the edges of the join graph.

(2) **Exploring Curated Join Paths.** After preprocessing, participants can browse the indexed join paths (Figure 4 ❷) through the interface. For each path, the system displays the estimated cardinality, attributes, hop count, and a visual representation where tables appear as nodes and joinable column pairs appear as edges (Figure 4 ❸).

(3) **Viewing Annotated Semantic Types.** During preprocessing, the LLM-annotated semantic types are persisted, allowing participants to inspect them via the interface (Figure 4 ❹).

(4) **Filtering Join Paths via Heatmap.** SemDisc provides a heatmap (Figure 4 ❺) that organizes join paths into bins by estimated tuple count (y-axis) and node count (x-axis). Each cell represents a bin of join paths satisfying both criteria, with darker shading indicating higher path counts.

(5) **Visualizing Hybrid Paths.** The system renders joinable edges within each path (Figure 4 ❻) using distinct colors: orange for semantic joins and blue for equi-joins.

## 3.2 Scenario 2: Answering Queries

In this scenario, participants load a data lake and submit a query table to retrieve the top-$K$ join paths satisfying their requirements:

(A) **Query Input.** SemDisc accepts queries structured as a query table (Figure 4 Ⓐ), where participants will be able to specify example values and/or semantic types of columns to generate a dataset of interest.

(B) **Top-$K$ Join Path Retrieval.** The system retrieves the top-$K$ join paths satisfying the query table. Participants can examine each path's structure (Figure 4 Ⓑ), view a tabular preview of that path, and download the materialized result of any join path as a CSV file (Figure 4 Ⓕ).

(C) **Highlighting Tuple Matches.** Beyond the heatmap navigation to fetch join paths based on estimated cardinality and table count, participants can select individual tuples from the query table

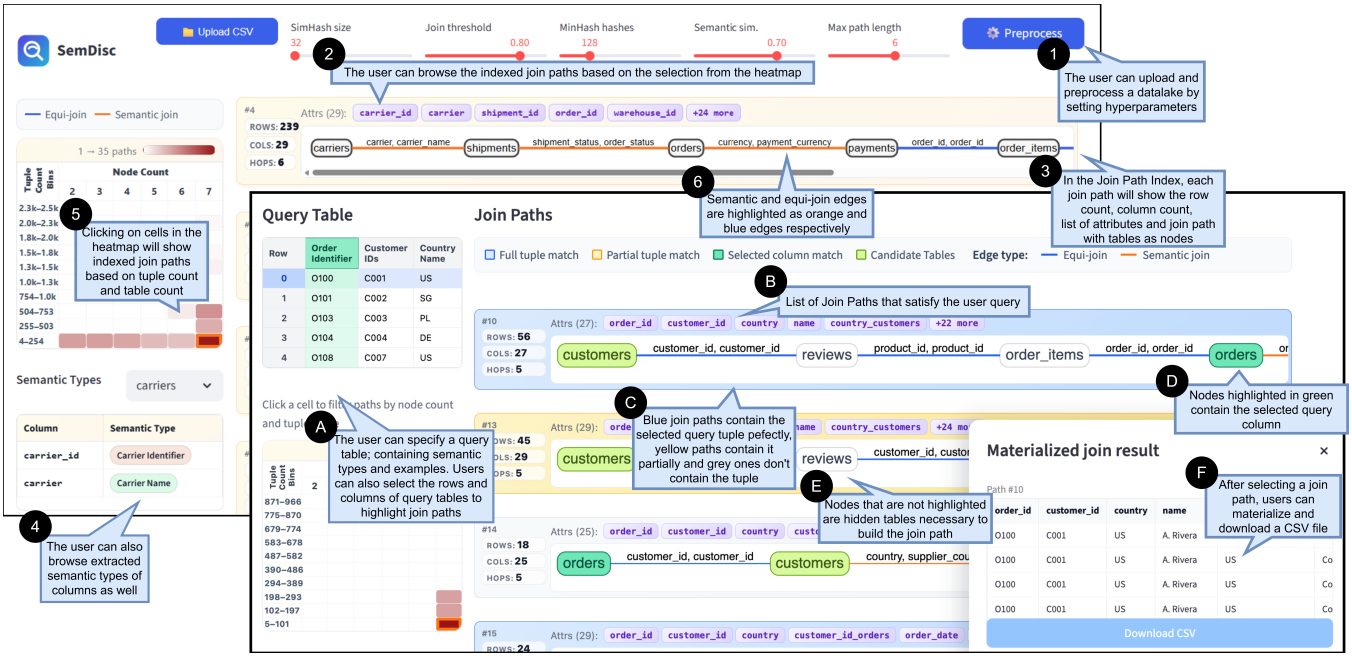


**Figure 4: User interface of SemDisc. Participants will be able to discover join paths from a data lake for any given query table.**

(Figure 4 A, highlighted in blue) to identify join paths that fully contain (Figure 4 C, blue paths), or partially contain (yellow paths) the user selected query tuple.

(D) **Highlighting Candidate Tables.** Participants can also select query columns (Figure 4 A, highlighted in green) to highlight candidate tables containing those columns within the join paths (Figure 4 D).

(E) **Detecting Hidden Tables.** Nodes that remain unhighlighted (Figure 4 E) represent *hidden tables*—tables sharing no example values or semantic types with the query columns yet essential for constructing complete join paths.

*Demonstration Engagement.* This demonstration enables participants to both visualize the results produced for a given query table and explore the offline indexing pipeline that powers SemDisc. Through an interactive interface, participants can modify the query table, adjust hyperparameters, and construct datasets of interest. By submitting different query tables, users can observe how hybrid join path discovery operates over data lakes.

## References


[1] [n. d.]. The Home of the U.S. Government's Open Data. https://data.gov/.

[2] [n. d.]. MITDWH. https://ist.mit.edu/warehouse.

[3] [n. d.]. SemDisc Demo Video. https://drive.google.com/drive/folders/18Uy6oJvgvnugKgs4h14jv57mmKeuaE9z?usp=sharing.

[4] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[5] A.Z. Broder. 1997. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No.97TB100171)*. 21–29. doi:10.1109/SEQUEN.1997.666900

[6] Tianji Cong, James Gale, Jason Frantz, H. V. Jagadish, and Çagatay Demiralp. 2023. WarpGate: A Semantic Join Discovery System for Cloud Data Warehouses. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. www.cidrdb.org.

[7] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, et al. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1925–1938.

[8] Till Döhmen, Radu Geacu, Madelon Hulsebos, and Sebastian Schelter. 2024. SchemaPile: A Large Collection of Relational Database Schemas. *Proceedings of the ACM on Management of Data* 2, 3 (2024).

[9] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2022. Deepjoin: Joinable table discovery with pre-trained language models. *arXiv preprint arXiv:2212.07588* (2022).

[10] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2023. DeepJoin: Joinable Table Discovery with Pre-Trained Language Models. *Proc. VLDB Endow.* 16, 10 (June 2023), 2458–2470. doi:10.14778/3603581.3603587

[11] Grace Fan, Roee Shraga, and Renée J Miller. 2024. Gen-T: Table Reclamation in Data Lakes. *arXiv preprint arXiv:2403.14128* (2024).

[12] Yue Gong, Zhiru Zhu, Sainyam Galhotra, and Raul Castro Fernandez. 2023. Ver: View discovery in the wild. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, ICDE, ICDE, 503–516.

[13] Yuxiang Guo, Yuren Mao, Zhonghao Hu, Lu Chen, and Yunjun Gao. 2025. Snoopy: Effective and Efficient Semantic Join Discovery via Proxy Columns . *IEEE Transactions on Knowledge & Data Engineering* 37, 05 (May 2025), 2971–2985. doi:10.1109/TKDE.2025.3545176

[14] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836. doi:10.1109/TPAMI.2018.2889473

[15] Mir Mahathir Mohammad and El Kindi Rezig. 2026. Qualitative Join Discovery in Data Lakes using Examples. *Proceedings of the ACM on Management of Data* 4, 1, Article 68 (mar 2026), 28 pages. https://mirmahathir.com/papers/semdisc.pdf

[16] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 3982–3992. doi:10.18653/v1/D19-1410

[17] El Kindi Rezig, Anshul Bhandari, Anna Fariha, Benjamin Price, Allan Vanterpool, Vijay Gadepally, and Michael Stonebraker. 2021. DICE: data discovery by example. *Proceedings of the VLDB Endowment* 14, 12 (2021), 2819–2822.